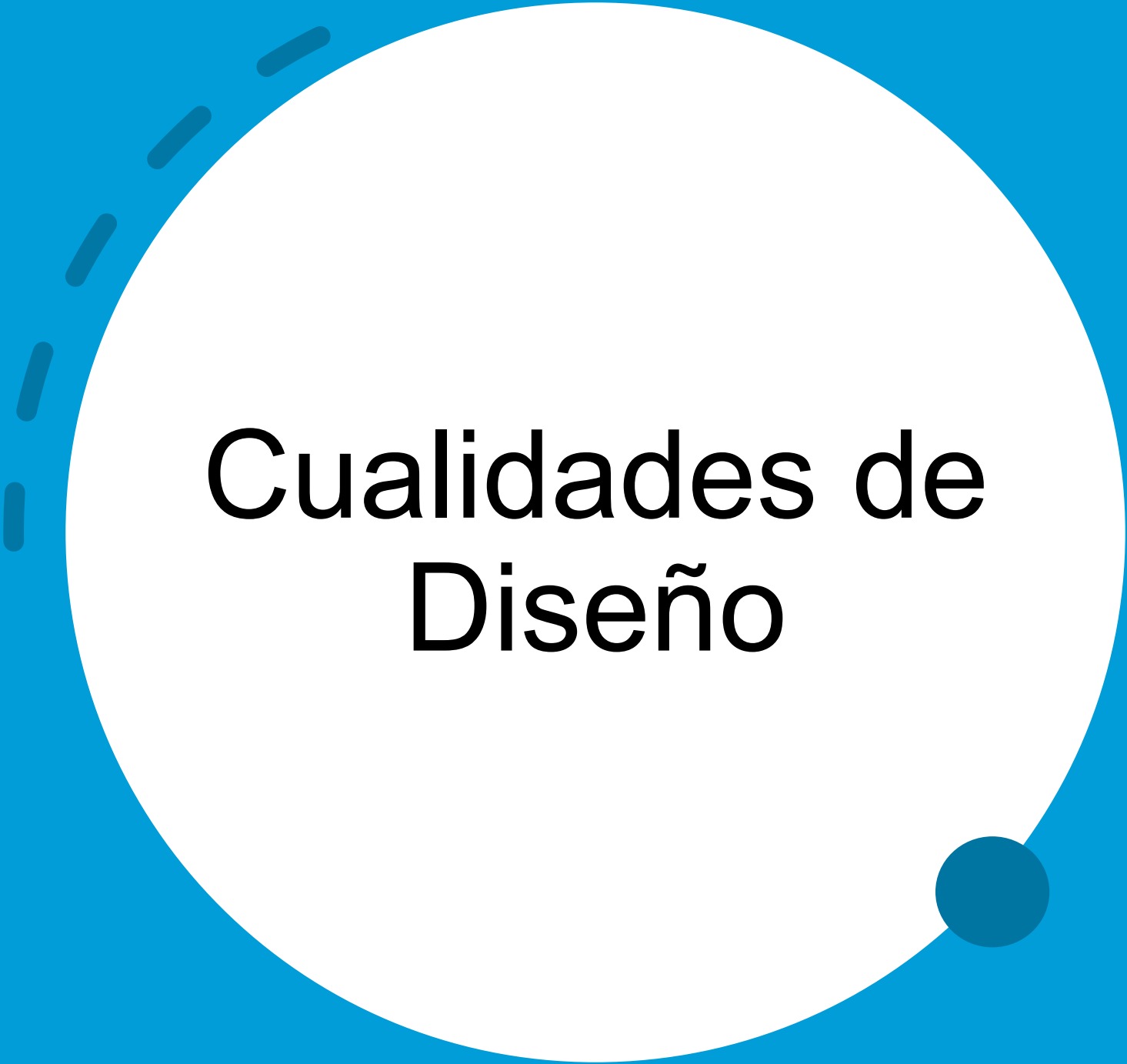


Agenda

- Cualidades de Diseño
- Code Smells
- Testing
- Test unitarios y su implementación con Jest
- Mocking

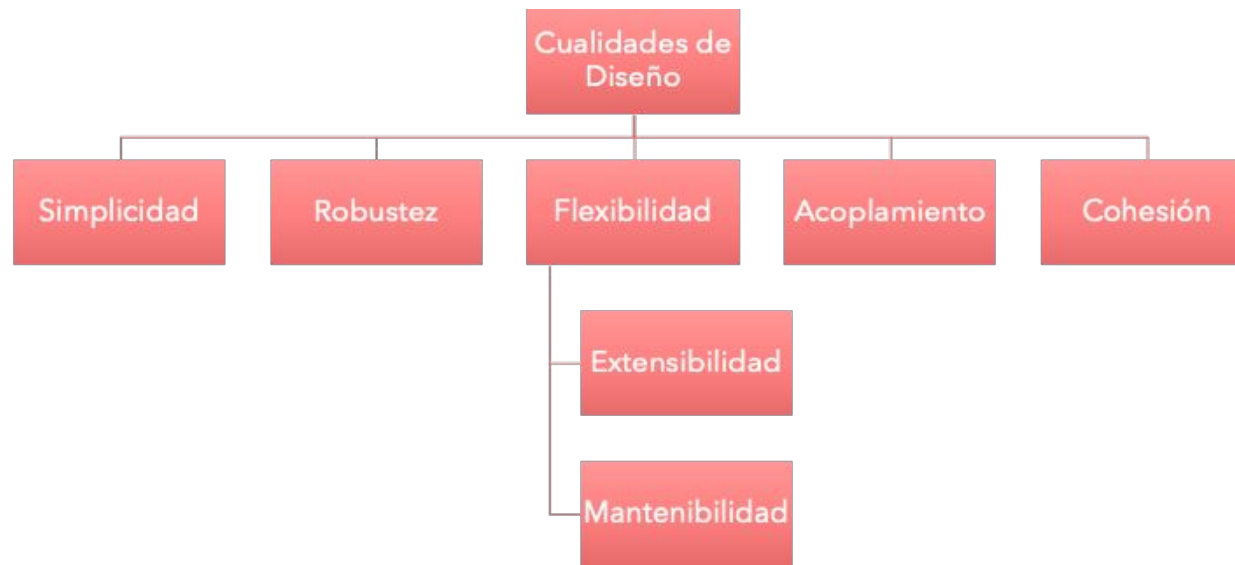
A large white circle is centered on a blue background. A dashed blue line starts from the top left and curves around the left side of the white circle. A solid dark blue circle is positioned at the bottom right edge of the white circle.

Cualidades de Diseño

Cualidades de Diseño

- Las Cualidades de Diseño sirven como criterios para analizar distintas propuestas/alternativas y tomar decisiones más formadas respecto a ellas.
- No son los únicos criterios: experiencia y conocimiento de quien esté realizando el proceso de diseño, también serán elementos determinantes.

Cualidades de Diseño



Acoplamiento

El acoplamiento se define como el grado de dependencia entre dos componentes, es decir, el grado de conocimiento que un componente tiene sobre el otro.

Cuanto mayor sea el acoplamiento entre dos componentes, los cambios o errores de uno de ellos impactarán en mayor medida sobre el otro componente.

Acoplamiento

Si minimizamos el acoplamiento podríamos:

- Mejorar la mantenibilidad
- Aumentar la reutilización
- Evitar que un defecto de un componente se propague a otros
- Evitar tener que inspeccionar y/o modificar múltiples componentes ante una modificación en uno solo de ellos

Cohesión

Un componente cohesivo tiende a tener todos sus elementos abocados a resolver el mismo problema.

La cohesión se define como la cantidad de responsabilidades que están asignadas a un componente.

Mientras más responsabilidades tenga un componente, menos cohesivo será.

Simplicidad

KISS – Keep it simple, stupid

Nos propone evitar cualquier complejidad innecesaria.

YAGNI – *You aren't gonna need it*

Nos propone no agregar funcionalidad nueva que no apunte a la problemática actual.

Robustez

Nos dice que ante un uso inadecuado por parte del usuario, sistemas externos o ante fallas internas:

- El sistema no debe generar información o comportamiento inconsistente/errático
- El sistema debe reportar los errores y volver a un estado consistente
- El sistema debe facilitar tanto como sea posible la detección de la causa del problema

Flexibilidad

Es la capacidad de reflejar cambios en el dominio de manera simple y sencilla.

- Extensibilidad: capacidad de agregar nuevas características con “poco” impacto.
- Mantenibilidad: capacidad de modificar las características existentes con “el menor” esfuerzo posible.



Principios SOLID

SOLID

Son principios básicos de Programación Orientada a Objetos y Diseño de Sistemas que nos ayudan a obtener mejores diseños implementando una serie de reglas o principios.

Nos ayudan a evitar la generación de “Código Sucio”

S - Single Responsibility Principle

- ***Una clase debe tener una única razón para cambiar.***
- Significa: Cada clase debe hacer una sola “cosa”.
- Formalmente: Buscamos generar clases altamente cohesivas.

S - Single Responsibility Principle

Ejemplo “malo”

```
class Report {  
    generate() {  
        // Genera el reporte  
    }  
  
    saveToFile() {  
        // Guarda el reporte en un archivo  
    }  
}
```

S - Single Responsibility Principle

Ejemplo “bueno”: Separamos la responsabilidad de la generación de la lógica de guardado.

```
class Report {  
    generate() {  
        // Genera el reporte  
    }  
}
```

```
class ReportSaver {  
    save(report) {  
        // Guarda el reporte en un archivo  
    }  
}
```

O - Open Closed Principle

- ***El código debe estar abierto para extensión, pero cerrado para modificación.***
- Significa: cuando agregamos nuevas funcionalidades, no deberíamos tener que cambiar el código existente, solo extenderlo.
- Formalmente: propiciar un diseño orientado a interfaces para aumentar la extensibilidad de nuestra solución.

O - Open Closed Principle

Ejemplo “malo”

```
function calculateArea(shape) {  
  if (shape.type === 'circle') {  
    return Math.PI * shape.radius * shape.radius;  
  } else if (shape.type === 'square') {  
    return shape.side * shape.side;  
  }  
}
```

O - Open Closed Principle

Ejemplo “bueno”: permitimos la creación de nuevas figuras con el cálculo de área sin tocar implementaciones existentes

```
class Shape {  
    area() {  
        throw new Error('Method not implemented');  
    }  
}  
  
class Circle extends Shape {  
    constructor(radius) {  
        super();  
        this.radius = radius;  
    }  
    area() {  
        return Math.PI * this.radius * this.radius;  
    }  
}
```

L - Liskov Substitution Principle

- ***Las subclases deben poder reemplazar a sus padres sin romper el programa.***
- Significa: Si un objeto de tipo Padre funciona en un programa, también debe funcionar perfectamente si se reemplaza por un objeto de tipo Hijo sin errores, excepciones ni resultados inesperados.
- Formalmente: Si S es subtipo de T, entonces los objetos de tipo T pueden ser reemplazados por objetos de tipo S sin afectar la corrección del programa (Principio de Sustitución).

L - Liskov Substitution Principle

Ejemplo “bueno”

```
abstract class BankAccount {  
    withdraw(amount) {  
        throw new Error('Method not implemented');  
    }  
}
```

L - Liskov Substitution Principle

Ejemplo “bueno”

```
class RegularAccount extends BankAccount {  
    constructor(balance) {  
        super();  
        this.balance = balance;  
    }  
  
    withdraw(amount) {  
        if (amount > this.balance) {  
            throw new Error('Insufficient funds');  
        }  
        this.balance -= amount;  
    }  
}
```

L - Liskov Substitution Principle

Ejemplo “bueno”

```
class OverdraftAccount extends BankAccount {  
  constructor(balance, overdraftLimit) {  
    super();  
    this.balance = balance;  
    this.overdraftLimit = overdraftLimit;  
  }  
  
  withdraw(amount) {  
    if (amount > this.balance + this.overdraftLimit) {  
      throw new Error('Overdraft limit exceeded');  
    }  
    this.balance -= amount;  
  }  
}
```

L - Liskov Substitution Principle

Ejemplo “bueno”

```
// Código que usa cuentas  
function makePayment(account, amount) {  
    account.withdraw(amount);  
}
```

L - Liskov Substitution Principle

Ejemplo “malo”

```
abstract class BankAccount {  
    withdraw(amount) {  
        throw new Error('Method not implemented');  
    }  
}
```

```
class LockedAccount extends BankAccount {  
    withdraw(amount) {  
        throw new Error('Account is locked');  
    }  
}
```


L - Liskov Substitution Principle

¿Por qué “malo”?

- BankAccount es una promesa, algo que debe ser completado por sus hijos (por eso lanza error).
- LockedAccount rompe la promesa: aunque cumple la interfaz (tiene withdraw), no respeta lo que el programa espera que haga un BankAccount.

I - Interface Segregation Principle

- ***Los clientes de un componente sólo deberían conocer de éste aquellos métodos que realmente usan y no aquellos que no necesitan usar.***
- Significa: No debemos forzar a una clase a implementar cosas que no necesita.
- Formalmente: Muchas interfaces cliente específicas son mejores que una interfaz de propósito general.

I - Interface Segregation Principle

Ejemplo “malo”

```
class Machine {  
    print() {}  
    scan() {}  
    fax() {}  
}  
  
class OldPrinter extends Machine {  
    print() {}  
    scan() {  
        throw new Error('Cannot scan');  
    }  
    fax() {  
        throw new Error('Cannot fax');  
    }  
}
```

I - Interface Segregation Principle

Ejemplo “bueno”: SimplePrinter solamente imprime. Lo que no necesita, no lo implementa.

```
interface Printer {  
    print() {}  
}  
  
interface Scanner {  
    scan() {}  
}  
  
class SimplePrinter implements Printer {  
    print() {  
        console.log('Printing...');  
    }  
}
```

D - Dependency Inversion Principle

- ***Depender de abstracciones, no de implementaciones concretas.***
- Significa: Los módulos de alto nivel no deben depender de módulos de bajo nivel. Ambos deben depender de abstracciones. Es una forma de desacoplar módulos.
- Formalmente: Una clase de mayor nivel de abstracción (“*de una capa superior*”) no debería depender de clases de bajo nivel de forma directa, sino que solamente debería utilizarlo a través de sus interfaces.

D - Dependency Inversion Principle

Ejemplo “malo”

```
class MySQLDatabase {  
    connect() {  
        console.log('Connecting to MySQL');  
    }  
}
```

```
class Application {  
    constructor() {  
        this.db = new MySQLDatabase();  
    }  
}
```

D - Dependency Inversion Principle

Ejemplo “bueno”: Application depende de una abstracción (Database), no le importa qué base de datos es.

```
class Database {  
    connect() {  
        throw new Error('Method not implemented');  
    }  
}  
  
class MySQLDatabase extends Database {  
    connect() {  
        console.log('Connecting to MySQL');  
    }  
}  
  
class Application {  
    constructor(db) {  
        this.db = db;  
    }  
}  
  
const app = new Application(new MySQLDatabase());
```



Code Smells

Code Smells

Los Code Smells, como su traducción nos dice, son “olores de código”.

- **Los smells** no necesariamente significan que esté mal lo que diseñé/implementé, sino que **son señales de algo que se puede mejorar** (*no hay que ser terminante, muchos code smells se entrecruzan al tratar de solucionarlos*).
- Nosotros debemos ser capaces de sentir estos “malos olores” en nuestro código para que, de esta forma, se pueda mejorar nuestro diseño.

Veamos algunos ejemplos...

Code Smells - Código Duplicado

- Qué es: Se repite el mismo código en varios lugares.
- Solución: Extraer a funciones o clases reutilizables.
- Indica: Que si hay que cambiar la lógica, podríamos olvidarnos de actualizar todos los lugares donde esté implementada esa misma lógica, generando bugs por inconsistencias.

Code Smells - Parámetros largos

- Qué es: Funciones con demasiados parámetros.
- Solución: Agrupar los parámetros en objetos o clases.
- Indica: Que cualquier cambio en la firma podría romper muchos objetos clientes que usan ese método.

Code Smells - Métodos largos

- Qué es: Métodos que hacen demasiado.
- Solución: Dividir en métodos más pequeños y claros.
- Indica: Que el método probablemente mezcla responsabilidades (*baja cohesión*) y es difícil de entender o testear (*baja mantenibilidad/testeabilidad*).

Code Smells - God Class

- Qué es: Clase que concentra demasiada responsabilidad.
- Solución: Dividir en varias clases especializadas.
- Indica: Que estamos violando el principio de responsabilidad única (SRP) y será difícil de mantener.

Code Smells - Herencia Rechazada

- Qué es: Subclases que no aprovechan la herencia.
- Solución: Usar composición o rediseñar la jerarquía.
- Indica: Que la jerarquía fue forzada y no respeta un modelo real de especialización.

Code Smells - Métodos fuera de lugar e intermediarios

- Qué es: Métodos que están en la clase equivocada o solo redirigen llamadas.
- Solución: Mover métodos a donde pertenezcan y eliminar intermediarios innecesarios.
- Indica: Que las clases están mal distribuidas y generan acoplamiento innecesario.

Testing

¿Qué es Testing?

Testing es la verificación dinámica de la adecuación del Sistema a los requerimientos/requisitos.

Según la IEEE, el testing es una actividad en la cual un Sistema o componente de software es ejecutado, bajo condiciones específicas, para que los resultados de dicha ejecución sean observados y/o registrados; y a partir de los mismos realizar una evaluación de algún aspecto del Sistema o componente.

Testing

- El testing busca encontrar fallas en el producto.
- Busca hacerlo lo más rápido y barato posible (busca la eficiencia).
- Busca encontrar la mayor cantidad de fallas.
- Intenta no detectar fallas que en realidad no lo son.
- Pretende encontrar las fallas más importantes.

Testing

Una prueba es exitosa si encuentra fallas

Testing

- Condiciones de prueba: son descripciones de situaciones que quieren probarse ante las cuales el Sistema debe responder.
- Casos de prueba: son lotes de datos necesarios para que se dé una determinada condición de prueba.



¡Gracias!